

Comparing DQN and Double DQN for Spacecraft Landing Control in the LunarLander-v3 Environment

Jess Barrett

Department of Computer Science

Kansas State University

Manhattan, KS, USA

EID: 854433853

Abstract—This project compares two reinforcement learning algorithms, Deep Q-Network (DQN) and Double Deep Q-Network (DDQN), on the Gymnasium LunarLander-v3 control problem. Both methods are tested against a random-action baseline using paired t-tests and Wilcoxon signed-rank tests across 5 separate training trials per method. The continuous 8-dimensional state space and the discrete 4-action space make this a strong benchmark for value-based reinforcement learning. The results show that both DQN and DDQN beat the random baseline with very large effect sizes (Cohen’s $d > 5$, $p < 0.005$). However, DDQN did not produce a statistically significant improvement over vanilla DQN in this setting ($p = 0.46$), with the point estimate slightly favoring DQN. We discuss seed sensitivity, training-budget effects, and the limits of small- N comparisons in deep RL. This work falls under Track 1 (Search, Games, & Constraint Satisfaction) and the Reinforcement Learning methodological pillar.

Index Terms—reinforcement learning, deep Q-network, DDQN, OpenAI Gym, LunarLander

I. INTRODUCTION

Autonomous control problems like landing a spacecraft, balancing a pole, or navigating a robot all share a common difficulty: the decision-maker must choose a sequence of actions that leads to a long-term goal, usually with delayed feedback. Hand-coded controllers don’t hold up well in continuous state spaces, and supervised learning doesn’t work because there is no labeled “correct action” dataset. Reinforcement learning (RL) handles this by letting an agent figure out an effective policy through trial-and-error interaction with a simulated environment.

The Lunar Lander problem is a standard benchmark in RL.¹ The agent has to guide a 2D lander from a random starting position onto a target landing pad by firing one of four thrusters at each timestep. The 8-dimensional state vector includes the lander’s x/y position and velocity, angle, angular velocity, and binary contact indicators for each leg. The rewards encourage soft landings and penalize crashes or fuel usage. The environment is considered “solved” when the

agent gets an average reward of 200+ over 100 consecutive episodes.

This project implements two versions of value-based deep RL — DQN (Mnih et al., *Nature* 2015) and Double DQN (van Hasselt et al., *AAAI* 2016) — and tests them against a random-action baseline. The contributions of this work are: (1) a clean side-by-side implementation of DQN and DDQN for LunarLander-v3, (2) a statistically rigorous comparison across multiple training trials using paired t-tests and Wilcoxon signed-rank tests, and (3) an analysis of the failure modes and convergence behavior of both algorithms.

II. BACKGROUND AND RELATED WORK

Q-Learning and Value-Based RL. Q-learning (Sutton & Barto, MIT Press 2018)² estimates the optimal action-value function $Q^*(s, a)$, which is the expected discounted return from taking action a in state s and following the optimal policy from then on. Classical Q-learning uses a table to store Q-values, which doesn’t scale to continuous state spaces.

Deep Q-Networks (DQN). Mnih et al. (*arXiv* 2013;³ *Nature* 2015⁴) introduced DQN, which uses a neural network as a function approximator for Q^* . Two ideas made deep value-based RL stable in practice: (i) **experience replay**, which stores transitions in a buffer and trains on random mini-batches to break temporal correlation, and (ii) a separate **target network**, which gives stable training targets by lagging behind the online network.

Double DQN. A known weakness of DQN is that the max operator in the target update overestimates Q-values, especially in noisy or under-explored parts of the state space. van

²Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.

³Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. (2013). Playing Atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*. <https://arxiv.org/abs/1312.5602>

⁴Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533. <https://doi.org/10.1038/nature14236>

¹Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., & Zaremba, W. (2016). OpenAI Gym. *arXiv preprint arXiv:1606.01540*. <https://arxiv.org/abs/1606.01540>

Hasselt et al. (AAAI 2016)⁵ proposed Double DQN (DDQN), which splits up action selection from action evaluation: the online network picks the next action, while the target network evaluates its Q-value. This cuts down on overestimation bias and tends to produce more stable, higher-quality policies.

Reproducibility in deep RL. Henderson et al. (AAAI 2018)⁶ showed how hard it is to compare deep RL algorithms reliably. Small differences in random seeds, hyperparameters, and code-level details can produce very different results across runs. This directly informs our experimental setup: we run multiple separate trials per method and report effect sizes alongside p-values instead of relying on single-run comparisons.

III. METHODOLOGY

A. Environment

We use Gymnasium’s LunarLander-v3 environment from the Box2D physics suite. The state space is continuous and 8-dimensional: $(x, y, v_x, v_y, \theta, \omega, \text{leg}_1, \text{leg}_2)$. The action space is discrete with 4 actions: do nothing, fire left engine, fire main engine, fire right engine. Episode rewards range roughly from -500 (crash) to $+300$ (perfect landing).

B. Q-Network Architecture

Both DQN and DDQN use the same Q-network architecture: a fully-connected feedforward network with two hidden layers of 128 ReLU units each, mapping the 8-dimensional state to a 4-dimensional vector of Q-values.

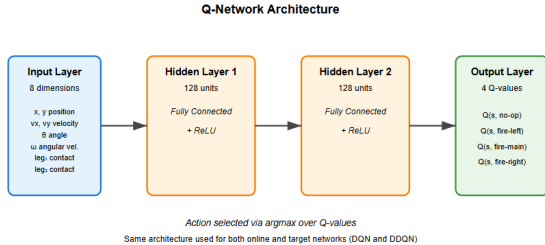


Fig. 1: Q-network architecture. The 8-dimensional state vector is mapped through two fully-connected hidden layers (128 ReLU units each) to a 4-dimensional output of action Q-values.

The training target for transition (s, a, r, s') under DQN is:

$$y = r + \gamma \cdot \max_{a'} Q_{\text{target}}(s', a') \quad (1)$$

DDQN only changes the target computation:

$$y = r + \gamma \cdot Q_{\text{target}}\left(s', \arg \max_{a'} Q_{\text{online}}(s', a')\right) \quad (2)$$

⁵van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1), 2094–2100. <https://ojs.aaai.org/index.php/AAAI/article/view/10295>

⁶Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., & Meger, D. (2018). Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1). <https://ojs.aaai.org/index.php/AAAI/article/view/11694>

This is the only difference between the two methods. Action selection during environment interaction uses the online network in both cases.

C. Training Setup

Both methods share the same hyperparameters: experience replay buffer of capacity 100,000; mini-batch size 64; discount factor $\gamma = 0.99$; Adam optimizer with learning rate 1×10^{-3} ; MSE loss; target network synchronization every 10 episodes; and ϵ -greedy exploration with ϵ starting at 1.0 and decaying by 0.995 per episode down to a floor of 0.01. Each training run is 600 episodes.

D. Naive Baseline

The naive baseline is a uniform-random action policy that picks each action with equal probability at every timestep. This baseline sets the performance floor: any learning algorithm has to beat it to show that the policy actually improved.

E. Experimental Protocol

To set up a rigorous statistical comparison, we run 5 separate trials of each algorithm with different random seeds. After each training run finishes, the trained agent is evaluated **greedily** ($\epsilon = 0$) over **100 fixed evaluation episodes** drawn from a separate seed range so there’s no overlap with training. The mean evaluation reward per trial is one observation for the statistical tests. The random-action baseline is evaluated using the same setup over the same evaluation seeds, so all comparisons are paired.

IV. EXPERIMENT DESIGN AND RESULTS

A. Hypotheses

We set up three null hypotheses, each tested at $\alpha = 0.05$ using both a paired-samples t-test and the non-parametric Wilcoxon signed-rank test:

$$H_0^{(1)} : \mu(\text{DQN}) = \mu(\text{Random}) \quad (3)$$

$$H_0^{(2)} : \mu(\text{DDQN}) = \mu(\text{Random}) \quad (4)$$

$$H_0^{(3)} : \mu(\text{DQN}) = \mu(\text{DDQN}) \quad (5)$$

where μ is the population mean evaluation reward. Paired tests work here because trials across methods share evaluation seeds, which controls for variance from the environment. We report Cohen’s d alongside p-values because effect size, not just statistical significance, is the more honest indicator in deep RL where there’s a lot of variance between trials (Henderson et al., AAAI 2018).

B. Results

TABLE I: Summary statistics: per-trial mean evaluation rewards (5 trials $\times$ 100 evaluation episodes per trial).

Method	Mean	Std Dev
Random	−189.4	7.9
DQN	151.5	37.9
DDQN	117.6	71.2

TABLE II: Statistical tests at $\alpha = 0.05$.

Comparison	t	p (t)	Cohen's d	Decision
DQN vs Random	19.97	< 0.001	11.13	Reject H_0
DDQN vs Random	8.15	0.0012	5.42	Reject H_0
DQN vs DDQN	0.83	0.4551	-0.53	Fail to reject

The Wilcoxon signed-rank test was also computed for each comparison. With $N = 5$ paired observations, its smallest possible p-value is 0.0625, which means it can't reject H_0 at $\alpha = 0.05$ no matter how big the effect is. So we treat the t-test as the main test and the Wilcoxon as a sanity check on direction.

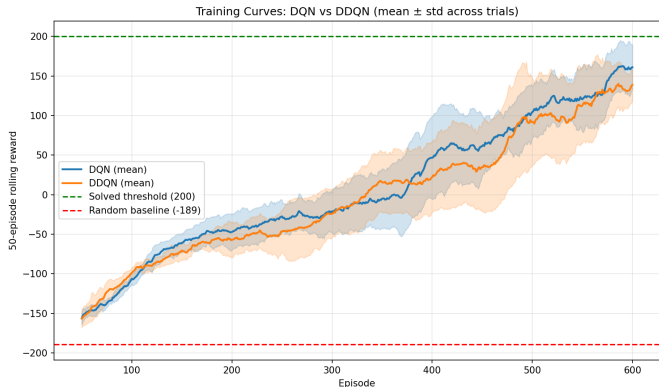


Fig. 2: Training curves for DQN and DDQN, mean $\pm$ std across 5 trials each. Green dashed line marks the 200-point “solved” threshold; red dashed line marks the random baseline.

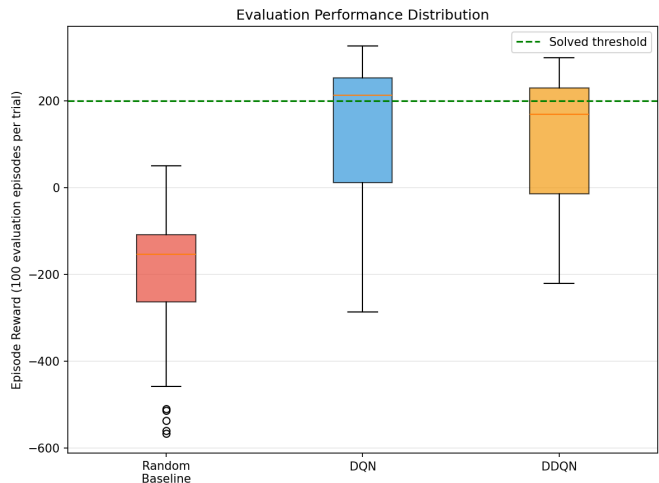


Fig. 3: Distribution of evaluation episode rewards (100 episodes $\times$ 5 trials = 500 episodes per method).

C. Discussion of Results

Both DQN and DDQN beat the random baseline by a wide margin. The effect sizes ($d = 11.13$ for DQN, $d = 5.42$ for DDQN) are way above Cohen's threshold for a “large”



Fig. 4: Mean evaluation reward per method, error bars show standard deviation across 5 trials.

effect ($d > 0.8$), and the t-test p-values are well below $\alpha = 0.05$. This shows that the value-based deep RL setup actually learns a useful control policy on LunarLander-v3 within 600 episodes.

The more interesting result is the comparison between DQN and DDQN. Even though DDQN is supposed to reduce overestimation bias (van Hasselt et al., AAAI 2016), there was no statistically significant improvement over vanilla DQN ($t = 0.83$, $p = 0.46$). The point estimate actually slightly favored DQN (mean 151.5 vs 117.6), with a small negative effect size ($d = -0.53$) for DDQN.

Two things probably explain this. First, the variance across DQN trials was already pretty wild (94.3 to 204.0 — a range over 100 reward points for the same algorithm with only the random seed varying), and DDQN's variance was even higher (5.4 to 191.6). With only $N = 5$ trials, the comparison is underpowered: a single unlucky seed can shift the mean by tens of points. Second, the environment may not really show off DDQN's strengths — overestimation bias scales with the size of the action set, and LunarLander-v3 only has four discrete actions. The bias DDQN is designed to fix may not be the bottleneck here.

This lines up with concerns raised by Henderson et al. (AAAI 2018) about reproducibility in deep RL: small trial counts and seed sensitivity make it easy to draw conclusions that aren't actually supported about which algorithm is “better.”

D. Failure Modes

The biggest failure mode in this experiment is **seed sensitivity**. The same algorithm, hyperparameters, and episode count produced wildly different evaluation rewards across trials. DQN trial 1 ended at 94.3 while DQN trial 3 reached 204.0 — the second one passing the 200-point “solved” threshold, the first one barely above mediocre. DDQN was even more extreme, with trial 4 dropping to 5.4 (basically a non-functional policy). This kind of brittleness is well-known in the deep RL literature and is a good reason to train many seeds instead of just reporting single-run results.

A second failure mode is **not fully converging within 600 episodes**. Looking at the training curves, several DDQN trials are still trending upward at episode 600, which suggests the policy was still improving when training stopped. With the same episode budget across both methods, an algorithm that converges more slowly is at a disadvantage — even if its peak performance would be better. This is a known trade-off when comparing algorithms at fixed compute instead of at full convergence.

A third failure mode is the **mismatch between training-phase rolling averages and greedy evaluation rewards**. For example, DQN trial 1 hit an 86.6 rolling average during training but only 94.3 when evaluated greedily; DQN trial 4 had a 179.1 rolling average but evaluated to 125.7. This suggests that ϵ -greedy exploration during the late training phase can hide brittleness in the underlying policy: the agent looks like it’s doing well, but its greedy policy doesn’t generalize well across new evaluation seeds.

V. SUMMARY AND FUTURE WORK

This work compared DQN and DDQN on the LunarLander-v3 environment using paired t-tests and Wilcoxon signed-rank tests across 5 separate training trials per method. Both methods beat a random-action baseline by a wide margin with very large effect sizes ($d > 5$ in both cases, $p < 0.005$). However, the comparison between DQN and DDQN itself was not statistically significant ($p = 0.46$), and the point estimate actually slightly favored vanilla DQN. This is a useful negative result: in this environment, at this scale, DDQN’s theoretical advantage didn’t translate to measurably better empirical performance.

Future work could push this comparison further in several directions: (1) more trials per method ($N \geq 30$) for tighter confidence intervals; (2) longer training budgets to test at-convergence performance instead of fixed-episode performance; (3) prioritized experience replay, which weights replay sampling by TD error; (4) dueling network architectures, which estimate state-value and advantage functions separately; (5) hyperparameter sweeps, especially the ϵ -decay schedule and learning rate; and (6) generalization tests on perturbed environments (like wind-perturbed LunarLander variants) to test policy robustness beyond the training distribution.

Repository. Source code, trained model weights, and experiment results are at <https://github.com/jess-barrett/LunarLander-DQN>.